
5 Concurrent Programming Techniques

CONTENTS

5.1 Task Management.....	105
5.2 Time and Delays.....	111
5.3 Semaphores.....	114
5.4 Message Passing.....	125
5.5 Summary.....	136

A side effect of the adoption of task-based scheduling is the need for task communication and synchronization. All operating systems invariably provide a rich selection of communication and synchronization primitives but, without a solid theoretical background, it is easy to misuse them and introduce subtle, time-dependent errors in the software.

This chapter provides the necessary background and bridges it into software development practice by presenting the main communication and synchronization primitives provided by the FREERTOS open-source operating system [18]. On the one hand, this RTOS is small/simple enough to be discussed in a small amount of space. On the other hand, it is comprehensive enough to give readers a thorough introduction to the topic.

Interested readers could refer to the FREERTOS reference manual [19], for more detailed information. The next chapter contains an introduction to real-time task scheduling algorithms and the related scheduling analysis theory for a few simple cases of practical relevance.

5.1 TASK MANAGEMENT

In all real-time operating systems, the concept of task as unit of scheduling plays a central role. Therefore, they all provide a set of primitives to instantiate new tasks and manage them, and FREERTOS is no exception. Table 5.1 summarizes the main task management and scheduling primitives provided by FREERTOS. To use them, it is necessary to include the main FREERTOS header file, `FreeRTOS.h`, followed by `task.h`.

In the rightmost column of the table, some primitives are marked as *optional*. This is because FREERTOS, like other operating systems designed for small em-

Table 5.1
Task Management Primitives of FREERTOS

Function	Purpose	Optional
vTaskStartScheduler	Start the scheduler	-
vTaskEndScheduler	Stop the scheduler	-
xTaskCreate	Create a new task	-
vTaskDelete	Delete a task given its handle	*
uxTaskPriorityGet	Get the priority of a task	*
vTaskPrioritySet	Set the priority of a task	*
vTaskSuspend	Suspend a specific task	*
vTaskResume	Resume a specific task	*
xTaskResumeFromISR	Resume a specific task from an ISR	*
xTaskIsTaskSuspended	Check whether a task is suspended	*
vTaskSuspendAll	Suspend all tasks but the running one	-
xTaskResumeAll	Resume all tasks	-
uxTaskGetNumberOfTasks	Return current number of tasks	-

bedded systems, can be configured to achieve different trade-offs between footprint and the richness of the operating system interface. Namely, a small set of operating system primitives is always provided, whereas others can be included or excluded from the configuration by defining appropriate macros in the system configuration file `FreeRTOSConfig.h`. Readers are referred to References [18, 19] for further information about FREERTOS configuration.

When using FREERTOS, all tasks share the same address space among them and with the operating system itself. This approach keeps the operating system as simple as possible, and makes it easy and efficient to share variables, because all tasks automatically and implicitly share memory. On the other hand, it becomes harder to protect tasks from each other with respect to illegal memory accesses, but it should be noted that many microcontrollers intended for embedded applications lack any hardware support for this protection anyway.

For some processor architectures, FREERTOS is able to use a memory protection unit (MPU), when available, to implement a limited form of memory access protection between tasks and the operating system itself. In this case, the operating system interface becomes slightly more complex because it becomes necessary to configure memory protection and specify how it applies to individual tasks. Even though a full discussion of this feature is outside the scope of this book, Chapter 15 contains more information about the inner workings of typical MPU hardware. Interested readers will find more information about software-related aspects in Reference [19].

Practically, the operating system is implemented as a library of object modules. Hence, as described in Chapter 3, the application program—comprising a set of tasks—is then linked against it when the application’s executable image is built, exactly as any other library. The application and the operating system modules are therefore bundled together in the resulting executable image.

For reasons more thoroughly discussed in Chapter 2, the executable image is usually stored in a nonvolatile memory within the target system (flash memories are in widespread use nowadays). When the system is turned on, the image entry point is invoked either directly, through the processor reset vector, or indirectly, by means of a minimal boot loader. When no boot loader is present, the executable image must also include an appropriate startup code, which takes care of initializing the target hardware before executing the `main()` C-language application entry point.

Another important aspect to remark on is that, when `main()` gets executed, the operating system scheduler is *not yet active*. It can be explicitly started and stopped by means of the following function calls:

```
void vTaskStartScheduler(void);
void vTaskEndScheduler(void);
```

It should be noted that the function `vTaskStartScheduler` reports errors back to the caller in an unusual way:

- When it is able to start the scheduler successfully, `vTaskStartScheduler` *does not return* to the caller at all because, in that case, execution proceeds with the tasks that have been created before starting the scheduler.
- On the other hand, `vTaskStartScheduler` may return to the caller for two distinct reasons, one normal and one abnormal:
 1. The scheduler was successfully started, but one of the tasks then stopped it by invoking `vTaskEndScheduler`.
 2. The scheduler was not started at all because an error occurred.

The two scenarios in which `vTaskStartScheduler` returns to the caller are clearly very different because, in the first case, the return is delayed and it usually occurs when the application is shut down in an orderly manner, for instance, at the user's request. On the contrary, in the second case the return is almost immediate and the application tasks are never actually executed.

However, since `vTaskStartScheduler` has no return value, there is no trivial way to distinguish between them. If the distinction is important for the application being developed, then the programmer must make the necessary information available on his or her own, for example, by setting a shared flag after a full and successful application startup so that it can be checked by the code that follows `vTaskStartScheduler`.

It is possible to create a new FreeRTOS task either before or after starting the scheduler, by calling the `xTaskCreate` function:

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    unsigned short usStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pvCreatedTask);
```

The only difference between the two cases is that, if a task is created when the scheduler is already running, it is eligible for execution immediately and may even preempt its creator. On the other hand, tasks created before starting the scheduler are not eligible for execution until the scheduler is started.

The arguments of `xTaskCreate` are defined as follows:

- `pvTaskCode` is a pointer to a function returning `void` and with one `void *` argument. It represents the entry point of the new task, that is, the function that the task will start executing from. This function must be designed to *never* return to the caller because this operation has undefined results in FREERTOS.
- `pcName`, a constant string of characters, represents the human-readable name of the task being created. The operating system simply stores this name along with the other task information it keeps track of, without interpretation, but it is useful when inspecting the operating system data structures, for example, as happens during debugging. The maximum length of the name actually stored by the operating system is limited by a configuration parameter; longer names are silently truncated.
- `usStackDepth` indicates how much memory must be allocated for the task stack. It is not expressed in bytes, but in machine *words*, whose size depends on the underlying hardware architecture. If necessary, the size of a stack word can be calculated by looking at the `StackType_t` data type, defined in the architecture-dependent part of FREERTOS itself.
- `pvParameters` is a `void *` pointer that will be passed to the task entry point, indicated by `pvTaskCode`, when task execution starts. The operating system does not interpret this pointer in any way. It is most commonly used to point to a shared memory structure that holds the task parameters and, possibly, its return values.
- `uxPriority` represents the initial, or baseline priority of the new task, expressed as a non-negative integer. The symbolic constant `tskIDLE_PRIORITY`, defined in the operating system's header files, gives the priority of the idle task, that is, the lowest priority in the system, and higher values correspond to higher priorities.
- `pvCreatedTask` points to the task *handle*, which will be filled if the task has been created successfully. The handle must be used to refer to the new task in the future by means of other operating system primitives.

The return value of `xTaskCreate` is a status code. If its value is `pdPASS`, the function was successful in creating the new task and filling the memory area pointed by `pvCreatedTask` with a valid handle, whereas any other value means that an error occurred and may convey more information about the nature of the error. The FREERTOS header `projdefs.h`, automatically included by `FreeRTOS.h`, contains the full list of error codes that may be returned by any operating system function.

For what concerns task execution, FREERTOS implements a *fixed-priority* scheduler in which, informally speaking, the highest-priority task ready for execution gets executed. As described in Sections 6.1 and 6.2, this scheduler is not only very intuitive and easy to understand, but also has several very convenient properties for real-time applications.

The total number of priority levels available for use is set in the operating system configuration. It must be determined as a trade-off between application requirements and overhead, because the size of several operating system data structures, and hence, the operating system memory requirements, depend on it. The currently configured value is available in the symbolic constant `configMAX_PRIORITIES`. Hence, the legal range of priorities in the system goes from `tskIDLE_PRIORITY` to `tskIDLE_PRIORITY+configMAX_PRIORITIES-1`, extremes included.

After creation, a task can be deleted by means of the function

```
void vTaskDelete(TaskHandle_t xTaskToDelete);
```

Its only argument, `xTaskToDelete`, is the handle of the task to be deleted. A task may also delete itself and, in this case, `vTaskDelete` will not return to the caller.

It is important to remark that, in FREERTOS, task deletion is *immediate* and *unconditional*. Therefore, the target task may be deleted and cease execution at any time and location in the code. This may lead to unforeseen side effects when, generally speaking, the task is holding some *resources* at the time of deletion. As a consequence, those resources will never be released and made available to other tasks again. See Section 5.3 for an example of this undesirable scenario.

Moreover, for technical reasons, the memory dynamically allocated to the task by the operating system (for instance, to store its stack) cannot be freed immediately during the execution of `vTaskDelete` itself; this duty is instead delegated to the idle task. If the application makes use of `vTaskDelete`, it is important to ensure that a portion of the processor time is available to the idle task. Otherwise, the system may run out of memory not because there is not enough, but because the idle task was unable to free it fast enough before it needed to be reused.

After creation, it is possible to change the priority of a task or retrieve it by means of the functions

```
void vTaskPrioritySet(TaskHandle_t xTask,
    UBaseType_t uxNewPriority);
```

```
UBaseType_t uxTaskPriorityGet(TaskHandle_t xTask);
```

Both functions take a task handle, `xTask`, as their first argument. The special value `NULL` can be used as a shortcut to refer to the calling task.

The function `vTaskPrioritySet` modifies the priority of a task after it has been created, and `uxTaskPriorityGet` returns the current priority of the task. It should, however, be noted that both the priority given at task creation and the priority set by `vTaskPrioritySet` represent the *baseline* priority of the task.

Instead, `uxTaskPriorityGet` returns the *active* priority of a task, which may differ from its baseline priority when the *priority inheritance* protocol for semaphore-based mutual exclusion, to be discussed in Sections 5.3 and 6.2, is in use.

The functions `vTaskSuspend` and `vTaskResume` take an argument of type `xTaskHandle` according to the following prototypes:

```
void vTaskSuspend(TaskHandle_t xTaskToSuspend);
void vTaskResume(TaskHandle_t xTaskToResume);
```

They are used to suspend and resume the execution of the task identified by their argument, respectively. In other words, these functions are used to move a task into and from state 4 of the task state diagram shown in Figure 4.8. For `vTaskSuspend`, the special value `NULL` can be used to suspend the invoking task, whereas, obviously, it makes no sense for a task to attempt to resume itself.

It should be noted that `vTaskSuspend` may suspend the execution of a task at an arbitrary point. Like `vTaskDelete`, it must therefore be used with care because any resources that the task is holding are not implicitly released while the task is suspended. As a consequence, any other tasks willing to access the same resources may have to wait until the suspended task is eventually resumed.

FREERTOS, like most other monolithic operating systems, does not hold a full task control block for interrupt handlers, and hence, they are not full-fledged tasks. One of the consequences of this design choice is that interrupt handlers cannot invoke many operating system primitives, in particular the ones that may block the caller. In other cases a specific variant of some primitives is provided, to be used specifically by interrupt handlers, instead of the normal one.

For this reason, for instance, calling `vTaskSuspend(NULL)` from an interrupt handler is forbidden. For related operating system design constraints, interrupt handlers are also not allowed to suspend regular tasks by invoking `vTaskSuspend` with a non-`NULL` `xTaskHandle` as argument. The function

```
UBaseType_t xTaskResumeFromISR(TaskHandle_t xTaskToResume);
```

is the variant of `vTaskResume` that must be used to resume a task from an interrupt handler, also known as interrupt service routine (ISR) in the FREERTOS nomenclature.

Since, as said above, interrupt handlers do not have a full-fledged, dedicated task context in FREERTOS, `xTaskResumeFromISR` cannot immediately and automatically perform a full context switch to a new task when needed, as its regular counterpart would do. An explicit context switch would be necessary, for example, when a low-priority task is interrupted and the interrupt handler resumes a higher-priority task.

On the contrary, `xTaskResumeFromISR` merely returns a nonzero value in this case, in order to make the invoking interrupt handler aware of the situation. In response to this indication, the interrupt handler must invoke the FREERTOS scheduling algorithm, as better discussed in Chapter 8, so that the higher-priority task just resumed will be considered for execution immediately after interrupt handling ends.

The *optional* function

```
UBaseType_t xTaskIsTaskSuspended(TaskHandle_t xTask);
```

can be used to tell whether or not a certain task, identified by its handle `xTask`, is currently suspended. Its return value is nonzero if the task is suspended, and zero otherwise.

The function

```
void vTaskSuspendAll(void);
```

suspends all tasks *but* the calling one. Symmetrically, the function

```
UBaseType_t xTaskResumeAll(void);
```

resumes all tasks suspended by `vTaskSuspendAll` and performs a context switch when necessary, for instance, when the priority of one of the resumed tasks is higher than the priority of the invoking task. In this case, the invoking task is later notified that it lost the processor for this reason because it will get a nonzero return value from `xTaskResumeAll`. Since the context switch is performed immediately within `xTaskResumeAll` itself, this function cannot be called from an interrupt handler.

Contrary to what could be expected, both `vTaskSuspendAll` and `xTaskResumeAll` are extremely efficient on single-processor systems, such as those targeted by FREERTOS. In fact, these functions are not implemented by suspending and resuming all tasks one by one—as their names would instead suggest—but by temporarily disabling the operating system scheduler. This operation requires little more work than updating a shared counter.

The last function related to task management simply returns the number of tasks currently present in the system, regardless of their states:

```
UBaseType_t uxTaskGetNumberOfTasks(void);
```

The count includes the calling task itself, as well as blocked and suspended tasks. Moreover, it may also include some tasks that have been deleted by `vTaskDelete`. This is a side effect of the delayed dismissal of the operating system's data structures associated with a task when the task is deleted.

5.2 TIME AND DELAYS

Most of the processing performed by a real-time system ought to be strongly correlated, quite intuitively, with the notion of *time*. Accordingly, FREERTOS provides a small set of primitives, listed in Table 5.2, which help tasks to *keep track* of the elapsed time and *synchronize* their activities with time, by delaying their execution. In order to use them, it is necessary to include `FreeRTOS.h`, followed by `task.h`.

More sophisticated timing facilities are optionally available in recent versions of the operating system, whose discussion is outside the scope of this book. For those, authoritative reference information can be found in Reference [19]. It should however be noted that the additional sophistication comes at a cost. First of all, they

Table 5.2
Time-Related Primitives of FREERTOS

Function	Purpose	Optional
xTaskGetTickCount	Return current time, in ticks	-
xTaskGetTickCountFromISR	Return current time, in ticks, to an ISR	-
vTaskDelay	Relative time delay	*
vTaskDelayUntil	Absolute time delay	*

consume more system resources, because their implementation is based on an additional system task, or *daemon*. Secondly, their interface is more complex and less intuitive.

In addition to the time-related primitives just introduced, all FREERTOS primitives that may potentially block the caller, such as those discussed in Sections 5.3 and 5.4, allow the calling task to specify an upper bound to the blocking time, and hence, avoid unpredictable disruptions to its execution schedule.

In FREERTOS the data type `TickType_t` is used to represent both the current time and a time interval. The current time is simply represented by the integer number of clock *ticks* elapsed from an arbitrary absolute time reference, that is, when the operating system scheduler was first started. As a consequence, the reference changes whenever the system is booted and the FREERTOS representation of time, by itself, is not enough to provide a correct notion of real-world, wall-clock time.

For this, applications must rely on dedicated hardware components like real-time clocks (RTC), which are fortunately widely available nowadays, even on very low-cost microcontrollers. As an added benefit, those components are usually able to keep track of wall-clock time even when the system is shut down, provided they are given an adequate backup power supply.

The length of a tick depends on the operating system configuration and, to some extent, on hardware capabilities. The configuration macro `configTICK_RATE_HZ` represents the configured tick frequency in Hertz and may be useful to convert back and forth between the usual time measurement units and clock ticks. For instance, the quantity `1000000/configTICK_RATE_HZ` is the tick length, approximated as an integer number of microsecond.

The function

```
TickType_t xTaskGetTickCount(void);
```

returns the current time, expressed as the integral number of ticks elapsed since the operating system scheduler was started. Barring low-level implementation details, FREERTOS maintains its notion of current time by incrementing a *tick counter* at the frequency specified by `configTICK_RATE_HZ`. This function simply returns the current value of the tick counter to the caller.

In most microcontrollers, the tick counter data type is either a 16- or 32-bit unsigned integer, depending on the configuration. Therefore, it is important to consider

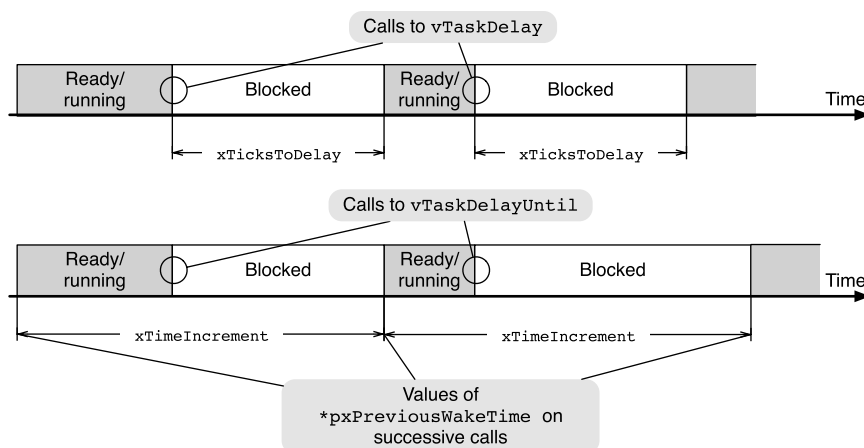


Figure 5.1 Relative versus absolute delays.

that the FREERTOS time counter will sooner or later wrap around and restart counting from zero. For instance, an unsigned, 32-bit counter incremented 1000 times per second—a common configuration choice for FREERTOS—will wrap around after about 1193 hours, that is, slightly more than 49 days.

It is therefore crucial that any application that must keep functioning correctly for a longer time without rebooting—as many real-time applications must do—is aware of the wraparound and handles it appropriately if it manipulates time values directly. If this is not the case, the application will be confronted with time values that suddenly “jump into the past” when a wraparound occurs, with imaginable consequences. The delay functions, to be discussed next, already handle time counter wraparound automatically, and hence, no special care is needed to use them.

Two distinct delay functions are available, depending on whether the delay should be *relative*, that is, measured with respect to the instant at which the delay function is invoked, or *absolute*, that is, until a certain instant in the future.

```
void vTaskDelay(const TickType_t xTicksToDelay);
```

```
void vTaskDelayUntil(TickType_t *pxPreviousWakeTime,
    const TickType_t xTimeIncrement);
```

- The function `vTaskDelay` implements a relative time delay. It blocks the calling task for `xTicksToDelay` ticks, then returns. As shown at the top of Figure 5.1, the time interval is relative to the time of the call, which is implicitly taken as a time reference point. The amount of delay from that reference point is fixed and is equal to `xTicksToDelay`.

- On the other hand, the function `vTaskDelayUntil` implements an absolute time delay. Referring to the bottom of Figure 5.1, the wake-up time for a certain invocation of the function is calculated as the wake-up time of the previous invocation, pointed by argument `pxPreviousWakeTime`, plus the time interval `xTimeIncrement`.

Hence, the wake-up time is unrelated to the function invocation time and the amount of delay introduced by `vTaskDelayUntil`, which varies from an invocation to another, so that the delay between wake-up events is kept constant and equal to `xTimeIncrement`, regardless of what the task did in between.

In extreme cases, the function might not block at all if the prescribed wake-up time is already in the past. Before returning, the function also increments the value pointed by `pxPreviousWakeTime` by `xTimeIncrement`, so that it is ready for the next call.

The right function to be used—and, as a consequence, the kind of delay introduced in task execution—depends on the purpose of the delay. A relative delay may be useful, for instance, if an I/O device must be allowed a certain amount of time to respond to a command. In this case, the delay must be measured from when the command has actually been sent to the device, and a relative delay makes sense.

On the other hand, an absolute delay is better when a task has to carry out an operation periodically because it guarantees that the period will stay constant even if the amount of time the task spends in the ready and running states—represented by the gray rectangles in Figure 5.1—varies from one instance to another.

5.3 SEMAPHORES

The first definition of a *semaphore* as a general intertask synchronization mechanism is due to Dijkstra [48]. The original proposal was based on busy wait, but most implementations found within contemporary operating systems use passive wait instead, without changing its semantics.

Over the years, semaphores have successfully been used to address many problems of practical significance in diverse concurrent programming domains even though, strictly speaking, they are not powerful enough to solve *every* concurrent programming problem that can be conceived [102].

Another reason of their popularity is that they are easy to implement in an efficient way. For this reason, virtually all operating systems offer semaphores as an intertask synchronization method.

According to its abstract definition, a semaphore is an object that contains two items of information, as shown in the right part of Figure 5.2:

- a *value* v , represented as a nonnegative integer, and
- a *queue* of tasks q , which are waiting on the semaphore.

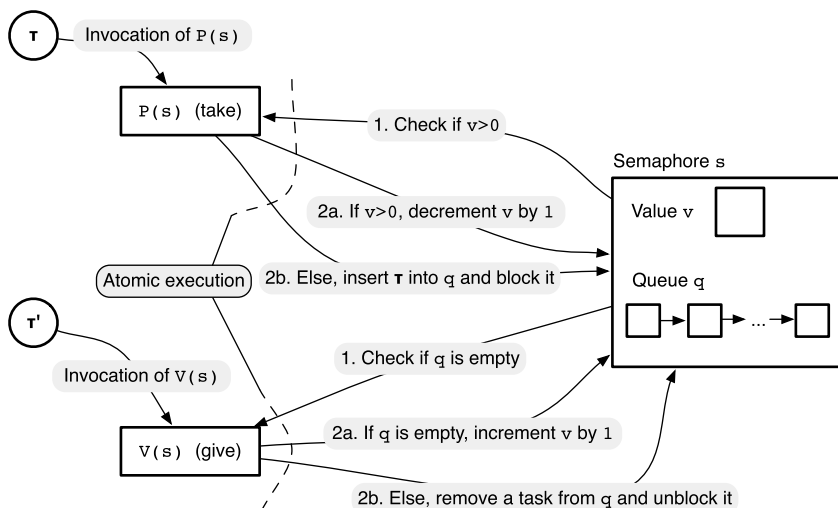


Figure 5.2 Abstract structure of a semaphore and behavior of its primitives.

The initial value of the semaphore is chosen (explicitly or implicitly) by the programmer upon semaphore creation, while the queue of waiting tasks is always initially empty.

Neither the value, nor the queue associated with a semaphore can be accessed or manipulated *directly* after initialization, albeit some implementation may make the current value of a semaphore available to programmers.

The only way to interact with a semaphore, and possibly modify its value and queue as a consequence, is to invoke the following abstract primitives, whose behavior is also depicted in Figure 5.2. An important assumption about those primitives is that their implementation ensures that they are executed *atomically*, that is, as indivisible units in which no context switches occur.

1. The primitive $P(s)$ —called *take* in the FREERTOS nomenclature—when invoked on a semaphore s checks whether or not the current value of s is strictly greater than zero.
 - If the value of s is strictly greater than zero, $P(s)$ decrements the value by one and returns to the caller. The calling task is not blocked in this case.
 - Otherwise, a reference to the calling task is inserted into the queue associated with s and the task is moved into the *Blocked* state of the task state diagram discussed in Section 4.5.
2. The primitive $V(s)$ —called *give* by FREERTOS—when invoked on a semaphore s checks whether or not the queue associated with s is currently empty.
 - If the queue is empty, $V(s)$ simply increments the value of s by one and returns to the caller. After the increment, the value of s will be strictly positive.

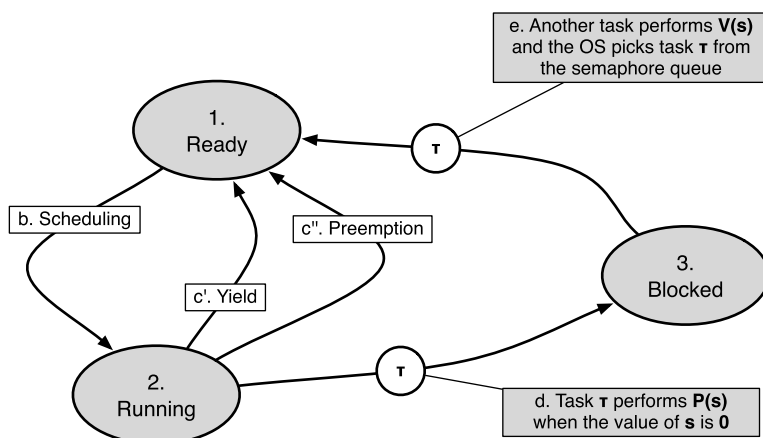


Figure 5.3 Task state diagram states and transitions involved in semaphore operations.

- Otherwise, the value of s is certainly zero. In this case, $V(s)$ picks one of the tasks referenced by the queue associated with s , removes the reference from the queue, and unblocks it. In other words, referring to the task state diagram, the selected task is moved back to the *ready* state, so that it is eligible for execution again.

It must be noted that semaphore primitives are tied to the task state diagram because their execution may induce the transition of a task from one state to another. Figure 5.3 is a simplified version of the full FREERTOS task state diagram (Figure 4.8) that highlights the nodes and arcs involved in semaphore operations.

As shown in the figure, the transition of a certain task τ from the *running* to the *blocked* state caused by $P(s)$ is voluntary because it depends on, and is caused by the invocation of the semaphore primitive by τ itself. On the contrary, the transition of τ back into the *ready* state is involuntary because it depends on an action performed by another task, namely, the task that invokes the primitive $V(s)$. It cannot be otherwise because, as long as τ is blocked, it cannot proceed with execution and cannot perform any action on its own.

Another aspect worth mentioning is that, when a task (τ in this case) is unblocked as a consequence of a $V(s)$, it goes into the *ready* state rather than *running*. Recalling how these states have been defined in Section 4.5, this means that the task is now *eligible* for execution again, but it does *not* imply that it shall *resume execution* immediately. This is useful to keep a proper separation of duties between the task synchronization primitives being described here and the task scheduling algorithms that will be presented in depth in Section 6.1. In this way:

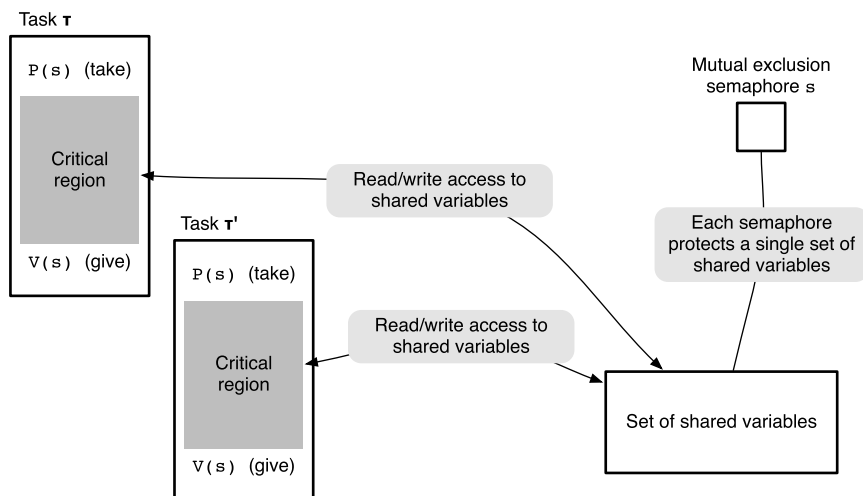


Figure 5.4 Usage of a semaphore and its primitives for mutual exclusion.

- The correct use of synchronization primitives stipulates whether or not a task may proceed in its execution, for instance, to avoid race conditions.
- The scheduling algorithms decide in which order tasks eligible to proceed are actually executed based on other criteria, for instance, their relative priority.

For similar reasons, the way semaphore queues are managed and, most importantly, *which* task is unblocked by $V(s)$ among those found in the semaphore queue, is left unspecified in the abstract definition of semaphore. This is because the queue management strategy must often be chosen in concert with the scheduling algorithm, in order to ensure that task synchronization and scheduling work well together. More information about this topic will be given in Section 6.2. The same considerations are also true for the message passing primitives, which embed a synchronization mechanism and will be the topic of Section 5.4.

A widespread application of semaphores is to ensure mutual exclusion among a number of critical regions that make access to the same set of shared variables, a key technique to avoid race conditions as described in Section 4.6. Namely, the critical regions are associated with a semaphore s , whose initial value is 1. The critical region's entry and exit code are the primitives $P(s)$ and $V(s)$. In other words, as shown in Figure 5.4, those primitives are placed like “brackets” around the critical regions they must protect.

A full formal proof of correctness of the mutual exclusion technique just described is beyond the scope of this book. However, its workings will be described in a simple case, in order to give readers at least a reasonable confidence that it is indeed correct. As shown in Figure 5.4, the example involves two concurrent tasks, τ and τ' , both

willing to enter their critical regions. Both critical regions are associated with the same set of shared variables and are protected by the same semaphore s .

- For the sake of the example, let us imagine that τ executes its critical region entry code (that is, the primitive $P(s)$) first. It will find that the value of s is 1 (the initial value of the semaphore), it will decrement the value to 0, and it will be allowed to proceed into its critical region immediately, without blocking.
- If any other tasks, like τ' in the figure, execute the primitive $P(s)$ they will be blocked because the current value of semaphore s is now 0. This is correct and necessary to enforce mutual exclusion, because task τ has been allowed to enter its critical region and it has not exited from it yet.
- It must also be noted that two or more executions of $P(s)$ cannot “overlap” in any way because they are executed atomically, one after another. Therefore, even though multiple tasks may invoke $P(s)$ concurrently, the operating system enforces an ordering among them to ensure that $P(s)$ indeed works as intended.
- Task τ' will be blocked at least until task τ exits from the critical region and executes the critical region exit code, that is, $V(s)$. At this point one of the tasks blocked on the semaphore and referenced by the semaphore queue, if any, will be unblocked.
- Assuming the choice fell on τ' , it will proceed when the operating system scheduling algorithm picks it up for execution, then τ' will enter its critical region. Mutual exclusion is still ensured because any tasks previously blocked on the semaphore are still blocked at the present time. Moreover, any additional tasks trying to enter their critical region and invoking a $P(s)$ will be blocked, too, because the semaphore value is still 0.
- When τ' exits from the critical region and executes $V(s)$ two different outcomes are possible:
 1. If some other tasks are still blocked on the semaphore, the process described above repeats, allowing another task into its critical region. In this case, the semaphore value stays at 0 so that any “new” tasks trying to enter their critical region will be blocked.
 2. If the queue associated with s is empty, that is, no tasks are blocked on the semaphore, $V(s)$ increments the value of s by one. As a result, the value of s becomes 1 and this brings the semaphore back to its initial state. Any task trying to enter its critical region in the future will be allowed to do so without blocking, exactly as happened to task τ in this example.

Another very important purpose of semaphores, besides mutual exclusion, is *condition synchronization*. With this term, we denote a form of synchronization in which a task must be blocked until a certain event happens or a condition is fulfilled.

For instance, let us consider an application in which a task τ_1 must transfer some information to another task, τ_2 . As shown in Figure 5.5, an intuitive way to

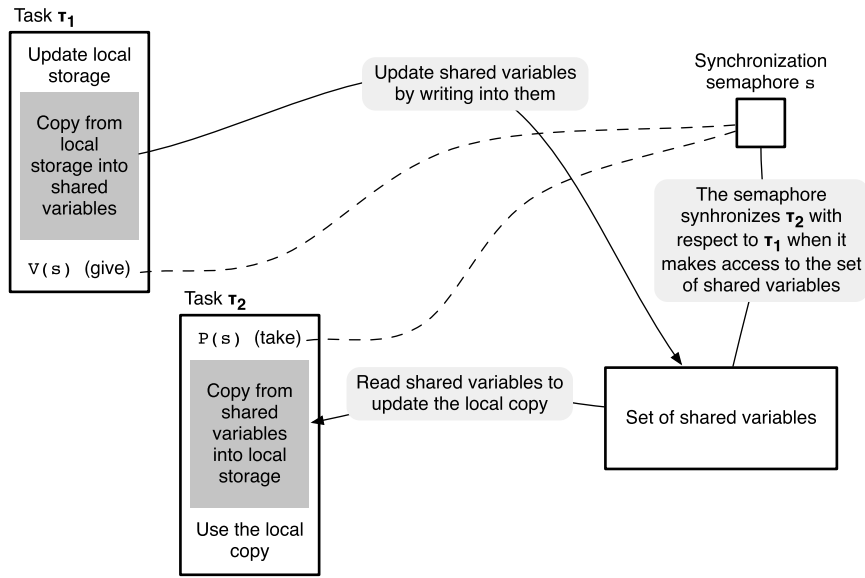


Figure 5.5 Usage of a semaphore and its primitives for condition synchronization.

implement the transfer is to allocate a set of shared variables large enough to hold the information to be transferred. Then, τ_1 writes the information into the shared variables after preparing it and τ_2 reads from the same variables, in order to create its local copy of the same information and use it.

Of course, τ_2 must synchronize with τ_1 so that it does not start reading from the shared variables before τ_1 has completely written into them the information to be transferred. If this is not the case, τ_2 may retrieve and use corrupted or inconsistent information. Namely, the goal is to *block* τ_2 until a certain condition is fulfilled. In this case, the condition is that the shared variables have been completely filled by τ_1 .

We neglect for the time being that τ_1 may be “too fast” for τ_2 , that is, we assume that τ_1 will never provide new information to be shared before τ_2 is done with the old one. Under this assumption the problem can be solved by using a *synchronization semaphore* s initialized to 0. Referring again to Figure 5.5, the semaphore is used by τ_1 and τ_2 in the following way:

- Before reading from the shared variables, τ_2 performs a $P(s)$. Since the initial value of the semaphore is 0 this primitive blocks τ_2 , unless τ_1 already filled the shared variables beforehand, as will be described in the following.
- After writing into the shared variables, τ_1 performs a $V(s)$. This primitive has two distinct outcomes depending on whether or not τ_2 already executed its $P(s)$ in the past:
 1. If τ_2 already executed its $P(s)$ and is currently blocked, it will be unblocked. Then, τ_2 is free to proceed and read from the shared variables.

2. If, informally speaking, τ_2 “is late” and did not execute its $P(s)$ yet, the value of s becomes 1. This signifies that the shared variables have been updated by τ_1 and are ready to be read by τ_2 in the future without further delay. In fact, when τ_2 will eventually execute $P(s)$, it will simply bring the value of s back to 0 without blocking.

As shown in the previous example, synchronization semaphores are usually more difficult to use correctly with respect to mutual exclusion semaphores, because there is no fixed template to follow. Namely, both the initial value of synchronization semaphores and the position of the primitives that operate on them within the code may vary widely, depending on the purpose of the semaphores.

Even though there is only one theoretical definition of semaphore, the FREERTOS operating system actually provides four different semaphore implementations, or “flavors.” This is a choice made by most operating systems, because each kind of semaphore represents a different trade-off between features and efficiency. It is the responsibility of the programmer to choose the best kind of semaphore case by case, bearing in mind which purpose a certain semaphore has in the system. The following paragraphs briefly describe the peculiarities of each kind of semaphore.

1. *Counting semaphores* are the most general kind of semaphore provided by FREERTOS. They are equivalent in most respects to the abstract semaphores discussed previously, but they are also the slowest. The main difference with respect to abstract semaphores is that, in the FREERTOS implementation, the maximum value that the semaphore value may legitimately assume must be declared in advance, when the semaphore is created.
2. *Binary semaphores* are less powerful than counting semaphores, because they have a maximum value as well as an initial value of one. As a consequence, their value can only be either one or zero, but they can still be used for either mutual exclusion or task synchronization. In return for this restriction, their implementation is somewhat faster than the one of counting semaphores.
3. *Mutex semaphores* are even more limited than binary semaphores, the additional restriction being that they must *only* be used as mutual exclusion semaphores. In other words, the $P(s)$ and $V(s)$ primitives on a mutex semaphore s must always appear in pairs and must be placed as brackets around critical regions. As a consequence, mutex semaphores cannot be used for task synchronization. In exchange for this, mutex semaphores implement *priority inheritance*. As discussed in Section 6.2, this is a key feature to guarantee a well-defined and acceptable interaction between task synchronization and scheduling from the point of view of real-time execution.
4. *Recursive mutex semaphores* have the same features as ordinary mutex semaphores. In addition, they also support the so-called *recursive* locks and unlocks, in which tasks are allowed to contain two (or more) nested critical regions, controlled by the same semaphore s and delimited by their own $P(s)$ and $V(s)$ brackets.

Table 5.3
Semaphore Creation/Deletion Primitives of FREERTOS

Function	Purpose	Optional
xSemaphoreCreateCounting	Create a counting semaphore	*
xSemaphoreCreateBinary	Create a binary semaphore	-
xSemaphoreCreateMutex	Create a mutex semaphore	*
xSemaphoreCreateRecursiveMutex	Create a recursive mutex	*
vSemaphoreDelete	Delete a semaphore (of any kind)	-

When s is an ordinary mutex semaphore, this arrangement leads to an indefinite wait because, when a task τ attempts to enter the inner critical region, it finds that the value of s is 0 and blocks. Clearly, this is an unwelcome side effect of the fact that τ itself previously entered the outer critical regions. In fact, as a general rule, τ blocks until the outer critical region is abandoned by the task that is currently executing within it. However, in this scenario that task is τ itself, and hence, τ will never be unblocked.

On the other hand, when s is a recursive mutex semaphore, the system prevents the issue by automatically taking and giving the semaphore only at the outermost critical region boundaries, as it should be.

The four kinds of semaphore just described are created by means of distinct functions, listed in Table 5.3. After creation, most kinds of semaphore (except recursive mutual exclusion semaphores) are used in a uniform way, by means of the same set of functions. In order to call any semaphore-related function, it is necessary to include two operating system headers, namely `FreeRTOS.h` and `semphr.h`. The first one contains general macros, data type definitions, and function prototypes pertaining to FREERTOS as a whole. The second one holds specific definitions related to semaphores.

The function

```
SemaphoreHandle_t xSemaphoreCreateCounting(
    UBaseType_t uxMaxCount,
    UBaseType_t uxInitialCount);
```

creates a counting semaphore with a given maximum (`uxMaxCount`) and initial (`uxInitialCount`) value. When successful, it returns to the caller a valid semaphore handle, a pointer to the internal operating system data structure representing the semaphore, of type `SemaphoreHandle_t`. If semaphore creation fails, the function returns a `NULL` pointer.

To create a binary semaphore, the function `xSemaphoreCreateBinary` should be used instead as:

```
SemaphoreHandle_t xSemaphoreCreateBinary(void);
```

Table 5.4
Semaphore Manipulation Primitives of FREERTOS

Function	Purpose	Optional
xSemaphoreTake	Perform a P() on a semaphore	-
xSemaphoreGive	Perform a V() on a semaphore	-
xSemaphoreTakeFromISR	P() from an interrupt handler	-
xSemaphoreGiveFromISR	V() from an interrupt handler	-
xSemaphoreTakeRecursive	P() on a recursive mutex	*
xSemaphoreGiveRecursive	V() on a recursive mutex	*

Like for `xSemaphoreCreateCounting`, the return value is either a valid semaphore handle upon successful completion, or a `NULL` pointer upon failure. Instead, since both the maximum and initial value of a binary semaphore are constrained to be 1, they are not explicitly indicated and the argument list is therefore empty. It should also be noted that binary semaphores are the only kind of semaphore that is always available for use in FREERTOS, regardless of how it has been configured. All the others are optional.

Mutual exclusion semaphores are created by means of two different functions, depending on whether the recursive lock and unlock feature is desired or not:

```
SemaphoreHandle_t xSemaphoreCreateMutex(void);
SemaphoreHandle_t xSemaphoreCreateRecursiveMutex(void);
```

Like the others, these creation functions also return either a valid semaphore handle upon successful completion, or a `NULL` pointer upon failure. All mutual exclusion semaphores are unlocked when they are first created—that is, their initial value is 1—and priority inheritance is always enabled for them.

A semaphore can be deleted by invoking the function:

```
void vSemaphoreDelete(SemaphoreHandle_t xSemaphore);
```

Its only argument is the handle of the semaphore to be deleted. This function must be used with care because the semaphore is destroyed immediately, even though there are some tasks waiting on it. Moreover, it is the programmer’s responsibility to ensure that a semaphore handle will never be used by any task after the corresponding semaphore has been destroyed, a constraint that may not be trivial to enforce in a concurrent programming environment.

After being created, semaphores are acted upon by means of the functions listed in Table 5.4. Most kinds of semaphore except recursive, mutual exclusion semaphores are acted upon by means of the functions `xSemaphoreTake` and `xSemaphoreGive`, the FREERTOS counterpart of P() and V(), respectively. As shown in the following prototypes, both take a semaphore handle `xSemaphore` as their first argument:

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore,
    TickType_t xBlockTime);
```

```
BaseType_t xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

Comparing the prototypes with the abstract definition of the same primitives, two differences are evident:

1. The abstract primitive $P()$ may block the caller for an unlimited amount of time. Since, for obvious reasons, this may be inconvenient in a real-time system, the function `xSemaphoreTake` has a second argument, `xBlockTime`, that specifies the maximum blocking time. In particular:
 - If the value is `portMAX_DELAY` (a symbolic constant defined when the main FREERTOS header file is included), the function blocks the caller until the semaphore operation is complete. In other words, when this value of `xBlockTime` is used, the concrete function behaves in the same way as its abstract counterpart.
For this option to be available, the operating system must be configured to support task suspend and resume, as described in Section 5.1.
 - If the value is 0 (zero), the function returns an error indication to the caller when the operation cannot be performed immediately.
 - Any other value is interpreted as the maximum amount of time the function will possibly block the caller, expressed as an integral number of clock *ticks*. See Section 5.2 for more information about time measurement and representation in FREERTOS.
2. According to their abstract definition, neither $P()$ nor $V()$ can ever fail. On the contrary, their real-world implementation may encounter an error for a variety of reasons. For example, as just described, `xSemaphoreTake` fails when a finite timeout is specified and the operation cannot be completed before the timeout expires. For this reason, the return value of `xSemaphoreTake` and `xSemaphoreGive` is a *status code*, which is `pdTRUE` if the operation was successful. Otherwise, they return `pdFALSE`.

Another important difference between the abstract definition of $P()$ and $V()$ with respect to their actual implementation is that, for reasons that will be better described in Chapter 8, neither `xSemaphoreTake` nor `xSemaphoreGive` can be invoked from an interrupt handler. In their place, the following two functions must be used:

```
BaseType_t xSemaphoreTakeFromISR(SemaphoreHandle_t xSemaphore,
    BaseType_t *pxHigherPriorityTaskWoken);
```

```
BaseType_t xSemaphoreGiveFromISR(SemaphoreHandle_t xSemaphore,
    BaseType_t *pxHigherPriorityTaskWoken);
```

Like all other FREERTOS primitives that can be invoked from an interrupt handler, these functions *never* block the caller. In addition, they return to the caller—in the variable pointed by `pxHigherPriorityTaskWoken`—an indication of whether or not they unblocked a task with a priority higher than the interrupted task.

The interrupt handler should use this information, as discussed in Chapter 8, to determine whether or not it should request the execution of the FREERTOS task scheduling algorithm before exiting. As before, the return value of these functions can be either `pdTRUE` or `pdFALSE`, depending on whether they were successful or not.

The last pair of special functions to be presented is the counterpart of `xSemaphoreTake` and `xSemaphoreGive` which must be used with recursive mutual exclusion semaphores:

```
BaseType_t xSemaphoreTakeRecursive(SemaphoreHandle_t xMutex,
    TickType_t xBlockTime);

BaseType_t xSemaphoreGiveRecursive(SemaphoreHandle_t xMutex);
```

Both their arguments and return values are the same as `xSemaphoreTake` and `xSemaphoreGive`, respectively.

One last aspect worth mentioning is the interaction between semaphore use and task deletion that, as discussed in Section 5.1 is immediate and unconditional. The high-level effect of deleting a task while it is *within* a critical region is therefore the same as the terminated task never exited from the critical region for some reason. Namely, no other tasks will ever be allowed to enter a critical region controlled by the same semaphore in the future.

Since this usually corresponds to a complete breakdown of any concurrent program, the direct invocation of `vTaskDelete` should usually be avoided, and it should be replaced by a more sophisticated deletion mechanism. One simple solution is to send a deletion *request* to the target task by some other means—for instance, one of the intertask communication mechanisms described in this section and the next one. The target task must be designed so that it responds to the request by terminating itself at a well-known location in its code after any required cleanup operation has been carried out and any mutual exclusion semaphore it held has been released.

Besides using semaphores, an alternative way to implement mutual exclusion is by means of the task suspend and resume primitives discussed in Section 5.1. In fact, in a single-processor system, `vTaskSuspendAll` opens a mutual exclusion region because the first task that successfully executes it will effectively prevent all other tasks from being executed until it invokes `xTaskResumeAll`.

As a side effect, any task executing between `vTaskSuspendAll` and `xTaskResumeAll` implicitly gets the highest possible priority in the system, except interrupt handlers. This consideration indicates that the method just described has two main shortcomings:

1. Any FREERTOS primitive that might block the caller for any reason even temporarily, or might require a context switch, must not be used within this kind of critical region. This is because blocking the only task allowed to run would completely lock up the system, and it is impossible to perform a context switch with the scheduler disabled.

2. Protecting critical regions with a sizable execution time in this way would probably be unacceptable in many applications because it leads to a large amount of unnecessary blocking. This is especially true for high-priority tasks, because if one of them becomes ready for execution while a low-priority task is engaged in a critical region of this kind, it will not run immediately, but only at the end of the critical region itself.

5.4 MESSAGE PASSING

The semaphore-based synchronization and communication methods discussed in Section 5.3 rely on two distinct and mostly independent mechanisms to implement synchronization and communication among tasks. In particular:

- Semaphores are essentially able to pass a *synchronization* signal from one task to another. For example, a semaphore can be used to block a task until another task has accomplished a certain action and make sure they operate together in a correct way.
- Data transfer takes place by means of *shared variables* because semaphores, by themselves, are unable to do so. Even though semaphores do have a value, it would be impractical to use it for data transfer, due to the way semaphores were conceived and defined.

Seen in a different way, the role of semaphores is to coordinate and enforce mutual exclusion and precedence constraints on task actions, in order to ensure that their access to shared variables takes place in the right sequence and at the appropriate time. On the other hand, semaphores are not actively involved in data transfer.

Message passing takes a radically different approach to task synchronization and communication by providing a *single* mechanism that is able, by itself, to provide both synchronization and data transfer at the same time and with the same set of primitives.

In this way, the mechanism not only works at a higher level of abstraction and becomes easier to use, but also it can be adopted with minimal updates when shared memory is not necessarily available. This happens, for example, in distributed systems where the communicating tasks may be executed by distinct computers linked by a communication network.

In its abstract form, a message passing mechanism is based upon two basic primitives, defined as follows:

- A *send* primitive, which transfers a certain amount of information, called a *message*, from one task to another. The invocation of the primitive may imply a synchronization action that blocks the caller if the data transfer cannot take place immediately.
- A *receive* primitive, which allows the calling task to retrieve the contents of a message sent to it by another task. Also in this case, the calling task blocks if the message it is seeking is not immediately available, thus synchronizing the receiver with the sender.

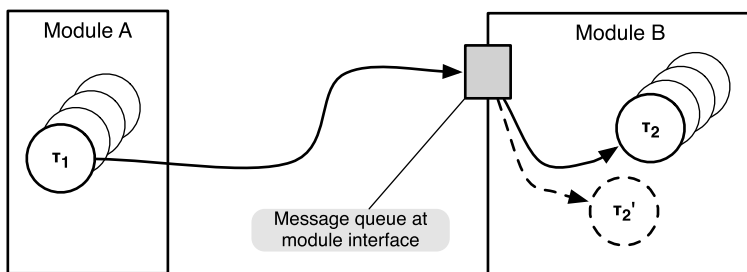


Figure 5.6 Indirect naming scheme in message passing.

Even if this definition still lacks many important lower-level details that will be discussed later, it is already clear that the most apparent effect of message passing primitives is to transfer a certain amount of information from the sending task to the receiving one. At the same time, message passing primitives also incorporate synchronization because they may block the caller when needed.

It should also be noted that, with message passing, mutual exclusion is not a concern because any given message is *never shared* among tasks. In fact, the message passing mechanism works as if the message were instantaneously copied from the sender to the receiver, so that message ownership is implicitly passed from the sender to the receiver when the message is transferred.

In this way, even if the sender modifies its local copy of a message after sending it, this will not affect the message already sent in the past. Symmetrically, the receiver is allowed to modify a message it received. This action is local to the receiver and does not affect the sender's local copy of the message in any way.

A key design aspect of a message passing scheme is how the sender identifies the intended recipient of a message and, symmetrically, how a receiver indicates which senders it is willing to receive messages from. In FREERTOS, the design is based upon an *indirect naming scheme*, in which the *send* and *receive* primitives are associated by means of a third, intermediate entity, known as *message queue*. Other popular names for this intermediate entity, used by other operating systems and in other contexts, are *mailbox* or *channel*.

As shown in Figure 5.6, even though adopting an indirect naming scheme looks more complex than, for instance, directly naming the recipient task when sending, it is indeed advantageous to software modularity and integration.

Let us assume that two software modules, *A* and *B*, each composed of multiple tasks and with a possibly elaborate internal structure, have to synchronize and communicate by means of message passing. In particular, a task τ_1 within module *A* has to send a message to a recipient within module *B*.

If we chose to directly name the recipient task when sending a message, then task τ_1 needs to know the internal structure of module *B* accurately enough to determine that the intended recipient is, for instance, τ_2 as is shown in the figure.

If the internal architecture of module B is later changed, so that the intended recipient becomes τ'_2 instead of τ_2 , module A must be updated accordingly. Otherwise, communication will no longer be possible or, even worse, messages may reach the wrong task.

On the contrary, if communication is carried out with an indirect naming scheme, module A and its task τ_1 must only know the name of the *message queue* that module B is using for incoming messages. Since the name of the mailbox is part of the interface of module B to the external world, it will likely stay the same even if the implementation or the internal design of the module itself changes over time.

An additional effect of communicating through a message queue rather than directly between tasks is that the relationship among communicating tasks becomes very flexible, albeit more complex. In fact, four scenarios are possible:

1. The simplest one is a *one-to-one* relationship, in which one task sends messages to another through the message queue.
2. A *many-to-one* relationship is also possible, in which multiple tasks send messages to a message queue, and a single task receives and handles them.
3. In a *one-to-many* relationship, a single task feeds messages into a message queue and multiple tasks receive from the queue. Unlike the previous two scenarios, this one and the following one cannot easily be implemented with direct inter-task communication.
4. The *many-to-many* relationship is the most complex one, comprising multiple sending and receiving tasks all operating on the same message queue.

Establishing a one-to-many or a many-to-many relationship does not allow the sending task(s) to determine exactly which task—among the receiving ones—will actually receive and handle its message. However, this may still be useful, for instance, to conveniently handle concurrent processing in software modules acting as servers.

In this case, those software modules will contain a number of equivalent “worker” tasks, all able to handle a single request at a time. All of them will be waiting for requests using the same message queue located at the module’s boundary. When a request arrives, one of the workers will be allowed to proceed and get it. Then, the worker will process the request and provide an appropriate reply to the requesting task. Meanwhile, the other workers will still be waiting for additional requests and may start working on them concurrently.

As mentioned earlier, message passing primitives incorporate both data transfer *and* synchronization aspects. Albeit the data transfer mechanism by itself is straightforward, in order to design a working concurrent application, it is important to look deeper into how message queue synchronization works.

As illustrated in Figure 5.7, message queues enforce two synchronization constraints, one for each side of the communication:

1. The receive primitive blocks the caller when invoked on an *empty* message queue. The invoking task will be unblocked when a message is sent to the queue. If multiple tasks are blocked trying to receive from a message queue when a message

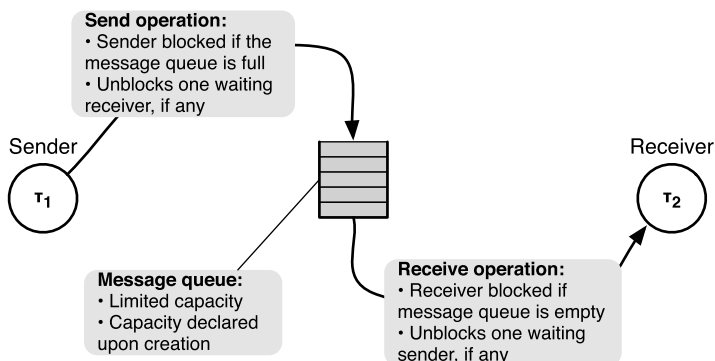


Figure 5.7 Message queue synchronization model.

arrives, the operating system selects and unblocks exactly one of them. That task receives the incoming message, whereas the others wait until further messages are sent to the queue.

2. The send primitive blocks the caller when the message queue it operates on is *full*, that is, the number of messages it currently contains is equal to its maximum capacity, declared upon creation. When a task receives a message from the queue, the operating system selects one of the tasks waiting to send, unblocks it, and puts its message into the message queue. The other tasks keep waiting until more space becomes available in the message queue.

It should also be noted that many operating systems (including **FreeRTOS**) offer a *nonblocking* variant of the send and receive primitives. Even though these variants may sometimes be useful from the software development point of view, they will not be further discussed here because, in that case, synchronization simply does not occur.

Another popular variant is a *timed* version of send and receive, in which it is possible to specify the maximum amount of time the primitives are allowed to block the caller. If the operation cannot be completed within the allotted time, the caller is unblocked anyway and the primitives return an error indication.

Under this synchronization model, considering the case in which the message queue is neither empty nor full, messages flow *asynchronously* from the sender to the receiver. No actual synchronization between those tasks actually occurs, because neither of them is blocked by the message passing primitives it invokes.

When necessary, stricter forms of synchronization can be implemented starting from these basic primitives. In particular, as shown in Figure 5.8:

- In a *synchronous* message transfer, often called *rendezvous*, the sending task τ_1 is blocked until the receiving task τ_2 is ready to receive the message. Moreover, as before, the receiving task is blocked by the receive primitive until a message is available.

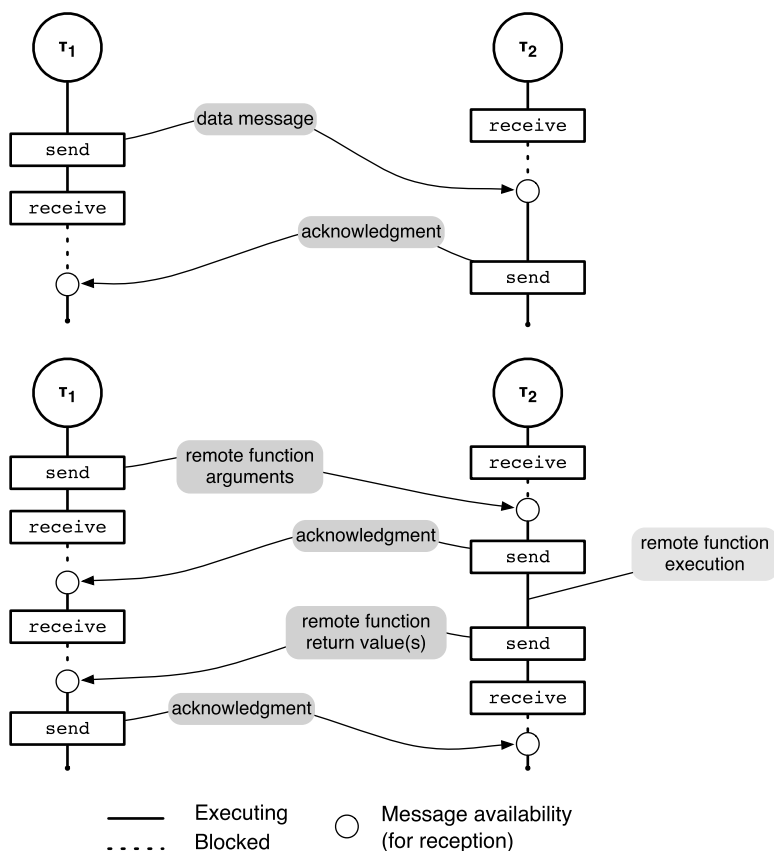


Figure 5.8 Synchronous and remote invocation transfers.

As shown on the top half of Figure 5.8, a synchronous transfer from τ_1 to τ_2 can be realized by means of two asynchronous message transfers going in opposite directions. The first transfer (from τ_1 to τ_2) carries the actual message to be transferred, and the second one (from τ_2 to τ_1) holds an acknowledgment.

A peculiarity of the second message transfer is that it is not used to actually move data between tasks—the transferred message may, in fact, be empty—but only for its synchronization semantics. Its purpose is to block τ_1 until τ_2 has successfully received the data message and subsequently sent the reply.

- A *remote invocation* message transfer, also known as *extended rendezvous*, enforces an even stricter synchronization between communicating tasks. Namely, when task τ_1 sends a remote invocation message to task τ_2 , it is blocked until a reply message is sent back from τ_2 to τ_1 . Symmetrically—

and here is the difference with respect to a synchronous transfer— τ_2 is blocked until that reply has been successfully received by τ_1 .

As the name suggests, this synchronization model is often used to imitate a function call, or invocation, using message passing. As in a regular function call, the requesting task τ_1 prepares the arguments of the function it wants task τ_2 to execute. Then, it puts them into a request message and sends the message to τ_2 , which will be responsible to execute the requested function. At the same time, τ_1 blocks and will not proceed further, until the function results become available.

Meanwhile, τ_2 has received the request and, through a local computation, executes the requested function and eventually generates the reply message. When the reply is ready, τ_2 sends it to τ_1 and unblocks it. It should be noted that at this stage τ_2 blocks until the reply has been received by τ_1 . In this way, τ_1 can make sure that the reply has reached its intended destination, or at least be notified if an error occurred.

As shown in the bottom half of Figure 5.8, a remote invocation from τ_1 to τ_2 can be implemented in terms of two synchronous message transfers going in opposite directions, that is, a total of four asynchronous message transfers.

Although, as has been shown previously, asynchronous message transfers can be used as a “building block” for constructing the most sophisticated ones—and are therefore very flexible—they have some drawbacks as well. As has been remarked, for instance, in Reference [25] the most important concern is probably that asynchronous message transfers give “too much freedom” to the programmer, somewhat like the “goto” statement of unstructured sequential programming.

The resulting programs are therefore more complex to understand and check for correctness, also due to the proliferation of explicit message passing primitives in the code. This observation leads us to remark on the importance of good programming practice to avoid using those message passing primitives directly if at all possible, but encapsulate them within higher-level communication functions, which implement more structured semantics.

The FREERTOS implementation of message queues follows very closely the abstract definition just given, with some added features tailored for real-time applications. Incidentally, FREERTOS makes use of message queues to implement semaphores, too. For this reason, all the primitives that operate on message queues, summarized in Table 5.5, are always available and cannot be removed from the operating system configuration. To use them, it is necessary to include the operating system headers `FreeRTOS.h` and `queue.h`.

The function `xQueueCreate` creates a new message queue and has the following prototype:

```
QueueHandle_t xQueueCreate(
    UBaseType_t uxQueueLength,
    UBaseType_t uxItemSize);
```

Table 5.5
Message Passing Primitives of FREERTOS

Function	Purpose	Optional
xQueueCreate	Create a message queue	-
vQueueDelete	Delete a message queue	-
xQueueSendToBack	Send a message	-
xQueueSend	Same as xQueueSendToBack	-
xQueueSendToFront	Send a high-priority message	-
xQueueOverwrite	Forcefully send	-
xQueueSendToBackFromISR	Send a message from interrupt handler	-
xQueueSendFromISR	Same as xQueueSendToBackFromISR	-
xQueueSendToFrontFromISR	Send a h.-p. message from interrupt handler	-
xQueueOverwriteFromISR	Forcefully send from interrupt handler	-
xQueueReceive	Receive a message	-
xQueueReceiveFromISR	Receive a message from interrupt handler	-
xQueuePeek	Nondestructive receive	-
xQueuePeekFromISR	Nondestructive receive from interrupt handler	-
uxQueueMessagesWaiting	Query queue length	-
uxQueueMessagesWaitingFromISR	Query queue length from interrupt handler	-
xQueueIsQueueEmptyFromISR	Check if a queue is empty from interrupt handler	-
xQueueIsQueueFullFromISR	Check if a queue is full from interrupt handler	-

Its two arguments specify the maximum number of elements the newly created message queue can contain, `uxQueueLength`, and the size of each element, `uxItemSize`, expressed in bytes. Upon successful completion, the function returns a valid message queue handle to the caller, which represents the message queue just created and must be used for any subsequent operation on it. If an error occurs, the function returns a `NULL` pointer instead.

When a message queue is no longer in use, it is useful to delete it, in order to reclaim the memory allocated to it for future use. This is done by means of the function:

```
void vQueueDelete (QueueHandle_t xQueue);
```

It should be noted that the deletion of a FREERTOS message queue takes place *immediately* and is never delayed, even though some tasks are currently blocked because they are engaged in a send or receive primitive on the message queue itself. The effect of the deletion on the waiting tasks depends on whether or not they specified a time limit for the execution of the primitive:

- if they did so, they will receive an error indication when a timeout occurs;
- otherwise, they will be blocked forever.

Since this kind of behavior—especially the second one—is often undesirable in a real-time system, it is the responsibility of the programmer to ensure that message

queues are deleted only when no tasks are blocked on them, when those tasks will no longer be needed in the future, or when they have other ways to recover (for instance, by means of a timeout mechanism).

After a message queue has been successfully created and its `xQueue` handle is available for use, it is possible to send a message to it by means of the functions

```
BaseType_t xQueueSendToBack (
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    TickType_t xTicksToWait);
```

```
BaseType_t xQueueSendToFront (
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    TickType_t xTicksToWait);
```

The first function, `xQueueSendToBack`, sends a message to the back of a message queue. The function `xQueueSend` is totally equivalent to `xQueueSendToBack` and has the same prototype. The message to be sent is pointed by the `pvItemToQueue` argument, whereas its size is implicitly assumed to be equal to the size of a message queue item, as declared when the queue was created.

The last argument, `xTicksToWait`, specifies the maximum amount of time allotted to the operation. The values it can assume are the same as for semaphore primitives. Namely:

- If the value is `portMAX_DELAY`, the behavior of the primitive is the same as its abstract counterpart. When the message queue is full, the function blocks the caller until the space it needs becomes available. For this option to be available, the operating system must be configured to support task suspend and resume, as described in Section 5.1.
- If the value is 0 (zero), the function returns an error indication to the caller when the operation cannot be performed immediately because the message queue is full at the moment.
- Any other value is interpreted as the maximum amount of time the function will possibly block the caller, expressed as an integral number of clock *ticks*. See Section 5.2 for more information about ticks and time measurement in FREERTOS.

The return value of `xQueueSendToBack` is `pdPASS` if the function was successful, whereas any other value means that an error occurred. In particular, the error code `errQUEUE_FULL` means that the function was unable to send the message within the maximum amount of time specified by `xTicksToWait` because the queue was full.

FREERTOS message queues are normally managed in first-in, first-out (FIFO) order. However, a high-priority message can be sent using the `xQueueSendToFront` function instead of `xQueueSendToBack`. The only difference between those two

functions is that `xQueueSendToFront` sends the message to the *front* of the message queue, so that it passes over the other messages stored in the queue and will be received before them.

Another possibility is to forcefully send a message to a message queue even though it is full by *overwriting* one message already stored into it in the past. This is generally done on message queues with a capacity of one, with the help of the following function:

```
BaseType_t xQueueOverwrite(
    QueueHandle_t xQueue,
    const void * pvItemToQueue);
```

Unlike for the previous functions, the argument `xTicksToWait` is not present because this function always completes its work immediately. As before, the return value indicates whether the function was successful or not.

Neither `xQueueSendToBack` nor `xQueueSendToFront` nor `xQueueOverwrite` can be invoked from an interrupt handler. Instead, either `xQueueSendToBackFromISR` or `xQueueSendToFrontFromISR` or `xQueueOverwriteFromISR` must be called in this case, according to the following prototypes:

```
BaseType_t xQueueSendToBackFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken);
```

```
BaseType_t xQueueSendToFrontFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken);
```

```
BaseType_t xQueueOverwriteFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken);
```

As before, the function `xQueueSendFromISR` is totally equivalent to `xQueueSendToBackFromISR`.

The differences of these functions with respect to their regular counterparts are:

- They never block the caller, and hence, they do not have a `xTicksToWait` argument. In other words, they always behave as if the timeout were 0, so that they return an error indication to the caller if the operation cannot be concluded immediately.
- The argument `pxHigherPriorityTaskWoken` points to a `BaseType_t` variable. The function will set the referenced variable to either `pdTRUE` or `pdFALSE`, depending on whether or not it awakened a task with a priority higher than the task which was running when the interrupt handler started.

The interrupt handler should use this information, as discussed in Chapter 8, to determine if it should invoke the FREERTOS scheduling algorithm before exiting.

Messages are always received from the front of a message queue by means of the following functions:

```
BaseType_t xQueueReceive(
    QueueHandle_t xQueue,
    void *pvBuffer,
    TickType_t xTicksToWait);
```

```
BaseType_t xQueuePeek(
    QueueHandle_t xQueue,
    void *pvBuffer,
    TickType_t xTicksToWait);
```

The first argument of these functions is a message queue handle `xQueue`, of type `QueueHandle_t`, which indicates the message queue they will work upon.

The second argument, `pvBuffer`, is a pointer to a memory buffer into which the function will store the message just received. The memory buffer must be large enough to hold the message, that is, at least as large as a message queue item.

The last argument, `xTicksToWait`, specifies how much time the function should wait for a message to become available if the message queue was completely empty when the function was invoked. The valid values of `xTicksToWait` are the same as already mentioned when discussing `xQueueSendToBack`.

The function `xQueueReceive`, when successful, *removes* the message it just received from the message queue, so that each message sent to the queue is received exactly once. On the contrary, the function `xQueuePeek` simply *copies* the message into the memory buffer indicated by the caller without removing it for the queue.

The return value of `xQueueReceive` and `xQueuePeek` is `pdPASS` if the function was successful, whereas any other value means that an error occurred. In particular, the error code `errQUEUE_EMPTY` means that the function was unable to receive a message within the maximum amount of time specified by `xTicksToWait` because the queue was empty. In this case, the buffer pointed by `pvBuffer` will not contain any valid message after these functions return to the caller and its contents shall not be used.

The functions `xQueueReceiveFromISR` and `xQueuePeekFromISR` are the variants of `xQueueReceive` and `xQueuePeek`, respectively, which must be used within an interrupt handler. Neither of them block the caller. Moreover, `xQueueReceiveFromISR` returns to the caller—in the variable pointed by `pxHigherPriorityTaskWoken`—an indication on whether or not it awakened a task with a higher priority than the interrupted one.

There is no need to do the same for `xQueuePeekFromISR` because it never frees any space in the message queue, and hence, it never wakes up any tasks. The two functions have the following prototype:

```

BaseType_t xQueueReceiveFromISR(
    QueueHandle_t xQueue,
    void *pvBuffer,
    BaseType_t *pxHigherPriorityTaskWoken);

```

```

BaseType_t xQueuePeekFromISR(
    QueueHandle_t xQueue,
    void *pvBuffer);

```

The functions belonging to the last group:

```

UBaseType_t
    uxQueueMessagesWaiting(const QueueHandle_t xQueue);

UBaseType_t
    uxQueueMessagesWaitingFromISR(const QueueHandle_t xQueue);

BaseType_t
    xQueueIsQueueEmptyFromISR(const QueueHandle_t xQueue);

BaseType_t
    xQueueIsQueueFullFromISR(const QueueHandle_t xQueue);

```

query various aspects of a message queue status. In particular,

- `uxQueueMessagesWaiting` and `uxQueueMessagesWaitingFromISR` return the number of items currently stored in the message queue `xQueue`. As usual, the latter variant must be used when the invoker is an interrupt handler.
- `xQueueIsQueueEmptyFromISR` and `xQueueIsQueueFullFromISR` return the Boolean value `pdTRUE` if the message queue `xQueue` is empty (or full, respectively) and `pdFALSE` otherwise. Both can be invoked safely from an interrupt handler.

These functions should be used with caution because, although the information they return is certainly accurate at the time of the call, the scope of its validity is somewhat limited. It is worth mentioning, for example, that the information may *no longer* be valid and should not be relied upon when any subsequent message queue operation is attempted because other tasks may have changed the queue status in the meantime.

For example, the preventive execution of `uxQueueMessageWaiting` by a task, with a result greater than zero, is not enough to guarantee that the same task will be able to immediately conclude a `xQueueReceive` with a non-zero `xTicksToWait` in the immediate future.

This is because other tasks, or interrupt handlers, may have received messages from the queue and emptied it completely in the meantime. On the contrary, `xQueueReceive` primitive, with `xTicksToWait` equal to 0, has been specifically designed to work as intended in these cases.

5.5 SUMMARY

This chapter contains an introduction to *concurrent programming* from the practical point of view, within the context of a real-world real-time operating system for embedded applications. Starting with the all-important concept of *task*, the basic unit of scheduling in real-time execution that has been discussed in Section 5.1, it was then possible to introduce readers to the main inter-task communication and synchronization mechanisms. This was the subject of Sections 5.3 and 5.4, which described the basic concept as well as the practical aspects of semaphore-based synchronization and message-passing, respectively.

Because this book focused on *real-time embedded systems*, rather than general-purpose computing, Section 5.2 also presented in detail how real-time operating systems manage time and timed delays and which primitives they make available to users for this purpose. The discussion precedes inter-process communication and synchronization because, in a real-time operating environment, the latter are obviously subject to timing constraints, too. These constraints are most often expressed by means of a timeout mechanism, which has many analogies with a timed delay.